Universidad Nacional de Ingeniería

Diseño de sistemas de Internet

Área del conocimiento de tecnología de la información y comunicación

Informe Final Diseño de sistemas en Internet

Sistema web de reservas del Complejo Deportivo Zacarías Guerra

Docente:

Msc. Cristopher Larios

Elaborado por:

Br. Garay Hernández Álvaro Josué — 2022-0800U

Br. Palacios Castillo Keneth Josué — 2022-0916U

Br. Muñoz Vivas Matthew Alexander — 2022-0806U

Managua, Nicaragua, 14 de noviembre de 2026

Índice

Introducción....4
Objetivos....4
Objetivo general....4
Objetivos específicos....4
Justificación y caso de estudio....5
Capítulo I: Análisis y diseño de sistemas web (UWE)....6
1.1 Actores y supuestos....6
1.2 Requerimientos funcionales....6
1.3 Requerimientos no funcionales....7
1.4 Casos de uso....8
1.5 Modelo de contenido....12
1.6 Modelo navegacional....14
1.7 Modelo de presentación....14
1.8 Modelo de procesos....15
1.9 Cambios respecto del Primer Entregable....19
Capítulo II: Arquitecturas de software para la web moderna....20
2.1 Arquitectura seleccionada....20
2.2 Arquitectura en capas, como se implementó....20
2.3 Ventajas y limitaciones observadas....22
2.4 Posible migración....22
Capítulo III: Análisis de la tecnología y del desarrollo....23
3.1 Tecnología utilizada....23
3.2 Patrón de diseño....24
3.3 Estrategias de desarrollo....25
3.4 Equipo y distribución del trabajo....26
3.5 Repositorio y versiones....26
Capítulo IV: Implementación del sistema....28
4.1 Modelo de datos....28
4.2 Reglas de negocio....29
4.3 API REST....29
4.4 Control de concurrencia....30
4.5 Seguridad....31
4.6 Interfaz web....31
4.7 Auditoría y notificaciones....35
Capítulo V: Contenedores, integración y despliegue continuos....36
5.1 Docker y Docker Compose....36
5.2 Integración continua....36

5.3 Despliegue continuo y producción....36
5.4 Relación con la guía del docente....37
Capítulo VI: Pruebas y validación....39
6.1 Pruebas automatizadas....39
6.2 Prueba integral contra producción....39
6.3 Tiempo de respuesta (RNF04)....39
6.4 Reservas simultáneas (RNF09)....40
6.5 Verificación de los requerimientos no funcionales....40
Capítulo VII: Trazabilidad y cumplimiento....41
7.1 Matriz de trazabilidad....41
7.2 Compromisos modificados....41
7.3 Compromisos omitidos y recortes....42
7.4 Cumplimiento de los objetivos....42
Conclusiones....44
Recomendaciones y trabajo futuro....45
Anexos....46
Anexo A. Enlaces y documentos....46

Introducción

El Complejo Deportivo Zacarías Guerra gestionaba sus reservas de forma manual, con el riesgo de dobles reservas y sin control sobre los pagos ni sobre las cancelaciones de último momento. Este informe presenta el sistema web de reservas desarrollado para el complejo durante el curso Diseño de Sistemas de Internet, y consolida el trabajo de los tres entregables anteriores con todo lo construido después de ellos.

El sistema entregado permite a los clientes registrarse, consultar la disponibilidad de las siete canchas, reservar con un pago simulado, cancelar con la penalización que corresponde y recibir notificaciones de cada cambio. El administrador gestiona canchas, horarios, reservas y usuarios, y consulta la auditoría de cada cambio de estado. El sistema está desplegado en un entorno público, con integración y despliegue continuos, y su código y documentación están en un repositorio de GitHub.

El informe describe el sistema tal como quedó implementado. Donde un entregable anterior decía una cosa y el sistema hace otra, se cuenta lo implementado y se señala el cambio con su motivo, de acuerdo con la indicación del docente de que todo compromiso modificado u omitido quede escrito y justificado.

El Capítulo I presenta el análisis y diseño con UWE, con los modelos corregidos. El Capítulo II revisa la arquitectura seleccionada a la luz de su implementación. El Capítulo III trata la tecnología, el patrón de diseño y la forma de trabajo del equipo. El Capítulo IV describe la implementación; el V, la contenedorización, la integración y el despliegue continuos; el VI, las pruebas y la validación; y el VII, la trazabilidad de cada compromiso. El informe cierra con conclusiones, recomendaciones y anexos.

Objetivos

Objetivo general

Desarrollar un sistema web de reservas para el Complejo Deportivo Zacarías Guerra, aplicando la metodología UWE y una arquitectura de software acorde al alcance del proyecto, desde el análisis de requerimientos hasta su despliegue en un entorno accesible.

Objetivos específicos

- Modelar los requerimientos funcionales y no funcionales del sistema mediante diagramas de casos de uso y el modelo de contenido de UWE.

- Diseñar el modelo navegacional y de presentación diferenciando las vistas del administrador y del cliente.
- Especificar el modelo de procesos, incluyendo el flujo de estados de una reserva y las reglas de penalización por cancelación tardía.
- Seleccionar una arquitectura de software backend/frontend acorde al alcance del proyecto.
- Implementar el backend con la lógica de negocio de disponibilidad, reservas y pagos.
- Implementar el frontend con interfaces diferenciadas por rol.
- Desplegar el sistema en un entorno accesible y documentar el proceso de despliegue y control de versiones.

Los objetivos específicos son los aprobados en el Primer Entregable. Su cumplimiento se evalúa en la sección 7.4.

Justificación y caso de estudio

Desde el punto de vista práctico, un sistema de este tipo resuelve un problema común en pequeños negocios deportivos de Nicaragua, donde la gestión informal de reservas genera pérdidas por dobles reservas o por falta de control de pagos. Desde el punto de vista académico, el proyecto aplica la metodología UWE de forma integral, desde los requerimientos hasta el modelo de procesos, y la lleva a una implementación real, cerrando el ciclo entre diseño e implementación. El dominio elegido, con reservas que cambian de estado y reglas de penalización, tiene la complejidad suficiente para producir modelos de proceso no triviales sin exceder el alcance de un proyecto de diez semanas.

El caso de estudio es el Complejo Deportivo Zacarías Guerra, dedicado exclusivamente al fútbol, con siete canchas: cinco de fútbol sala y dos de fútbol campo. Toda operación de pago es simulada: el sistema registra los cargos y reembolsos, pero no mueve dinero real. Las reglas de negocio que dieron forma al sistema son las del supuesto del Primer Entregable:

- Una cancelación con menos de 24 horas de anticipación pierde el 50 % del monto pagado.
- Si el cliente no se presenta sin haber cancelado (no-show), pierde el 100 %.

Capítulo I: Análisis y diseño de sistemas web (UWE)

Este capítulo presenta los modelos UWE del sistema. Los requerimientos se conservan como se aprobaron en el Primer Entregable. Los modelos de casos de uso, contenido, procesos y navegación se presentan en su versión corregida: durante la implementación se detectaron puntos donde el diagrama contradecía lo decidido o lo construido, y se corrigieron en el repositorio (carpeta `docs/modelos/`) para que modelo y sistema coincidan. La sección 1.9 resume esos cambios.

1.1 Actores y supuestos

El sistema tiene dos actores humanos:

- **Cliente:** consulta la disponibilidad, reserva, paga de forma simulada, cancela sus reservas y consulta su historial y sus notificaciones.
- **Administrador:** gestiona canchas, horarios, reservas y usuarios, y consulta la auditoría.

En los modelos corregidos aparece además el actor **Sistema**, responsable de marcar como no-show las reservas cuyo horario pasó sin que se registrara la asistencia. El supuesto central es la regla de penalización descrita en el caso de estudio.

1.2 Requerimientos funcionales

La Tabla 1 lista los diecisiete requerimientos funcionales del Primer Entregable. La columna de estado resume lo registrado en la matriz de trazabilidad (Capítulo VII).

Tabla 1. Requerimientos funcionales

#	Descripción	Actor	Estado
RF01	El sistema debe permitir al cliente registrarse.	Cliente	Cumplido
RF02	El sistema debe permitir al cliente consultar disponibilidad de canchas por fecha y horario.	Cliente	Cumplido
RF03	El sistema debe permitir al cliente crear una reserva sobre un horario disponible.	Cliente	Cumplido
RF04	El sistema debe permitir al cliente cancelar una reserva propia.	Cliente	Cumplido
RF05	El sistema debe aplicar automáticamente la penalización correspondiente si la cancelación es tardía o no hay presentación.	Sistema	Cumplido
RF06	El sistema debe simular el pago de la reserva y actualizar su estado a "pagada".	Cliente / Sistema	Modificado
RF07	El sistema debe permitir al cliente visualizar el historial de	Cliente	Cumplido

#	Descripción	Actor	Estado
	sus propias reservas.		
RF08	El sistema debe permitir al administrador registrar, editar y dar de baja canchas. La baja es lógica; si existen reservas activas asociadas, el sistema alerta al administrador y bloquea la baja salvo confirmación explícita.	Administrador	Cumplido
RF09	El sistema debe permitir al administrador definir los horarios disponibles por cancha.	Administrador	Cumplido
RF10	El sistema debe permitir al administrador visualizar todas las reservas del complejo, filtradas por fecha, cancha o estado.	Administrador	Cumplido
RF11	El sistema debe permitir al administrador confirmar o cancelar manualmente una reserva.	Administrador	Cumplido
RF12	El sistema debe notificar al cliente el cambio de estado de su reserva.	Sistema	Cumplido
RF13	El sistema debe registrar en auditoría los cambios de estado de cada reserva y de cada cancha (usuario responsable, fecha, motivo).	Sistema	Cumplido
RF14	El sistema debe restringir el acceso a las funciones de administración solo al rol Administrador.	Sistema	Cumplido
RF15	El sistema debe permitir a un usuario registrado iniciar sesión con sus credenciales.	Cliente / Administrador	Cumplido
RF16	El sistema debe permitir a cualquier usuario autenticado cerrar sesión desde el menú principal de su rol.	Cliente / Administrador	Cumplido
RF17	El sistema debe permitir a un Administrador gestionar usuarios.	Administrador	Cumplido

RF06 figura como modificado porque no existe un paso posterior de "actualizar a pagada": la reserva nace en estado `PAGADA`, ya que el pago simulado ocurre dentro de la misma transacción que la crea.

1.3 Requerimientos no funcionales

Tabla 2. Requerimientos no funcionales

#	Descripción	Categoría
RNF01	La interfaz debe ser clara y usable sin necesidad de capacitación previa.	Usabilidad
RNF02	Las contraseñas deben almacenarse cifradas (hash), nunca en	Seguridad

#	Descripción	Categoría
	texto plano.	
RNF03	El acceso a funcionalidades debe estar restringido según el rol del usuario.	Seguridad
RNF04	Las consultas de disponibilidad deben responder en menos de 3 segundos bajo uso normal.	Rendimiento
RNF05	El frontend debe ser responsive (usable desde celular).	Compatibilidad
RNF06	El backend y el frontend deben estar organizados en capas separadas para facilitar el trabajo en equipo.	Mantenibilidad
RNF07	El sistema debe poder desplegarse en un entorno accesible públicamente para la demo/defensa.	Portabilidad
RNF08	El diseño de datos debe permitir agregar más canchas o modalidades de juego sin rediseño mayor.	Escalabilidad
RNF09	El sistema debe garantizar mediante integridad transaccional que dos reservas simultáneas no puedan ocupar el mismo horario de la misma cancha.	Integridad / Concurrencia

Los nueve requerimientos no funcionales se cumplieron. La forma en que se verificó cada uno se detalla en la sección 6.5.

1.4 Casos de uso

La Figura 1 reproduce el diagrama de casos de uso del negocio del Primer Entregable, que no requirió correcciones: resume las tres necesidades del complejo (reservar y cobrar, cancelar y penalizar, y gestionar la disponibilidad) y los requerimientos que las cubren.

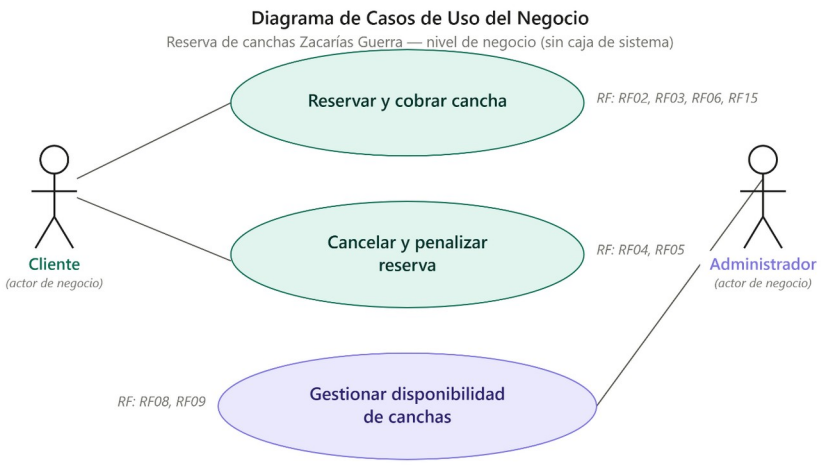


Figura 1. Casos de uso del negocio (Primer Entregable)

La Figura 2 muestra el diagrama general de casos de uso del sistema, corregido. Respecto del Primer Entregable, Simular pago pasa a ser una inclusión obligatoria de Reservar cancha, porque el pago siempre ocurre; Registrar usuario deja de asociarse al Administrador, porque el registro público crea solo clientes; y se agregan tres casos que el sistema implementa y el diagrama original no mostraba: Registrar asistencia, Marcar no-show (con el actor Sistema) y Registrar auditoría. Reservar, confirmar, cancelar, registrar asistencia y marcar no-show incluyen la notificación al cliente y el registro en auditoría.

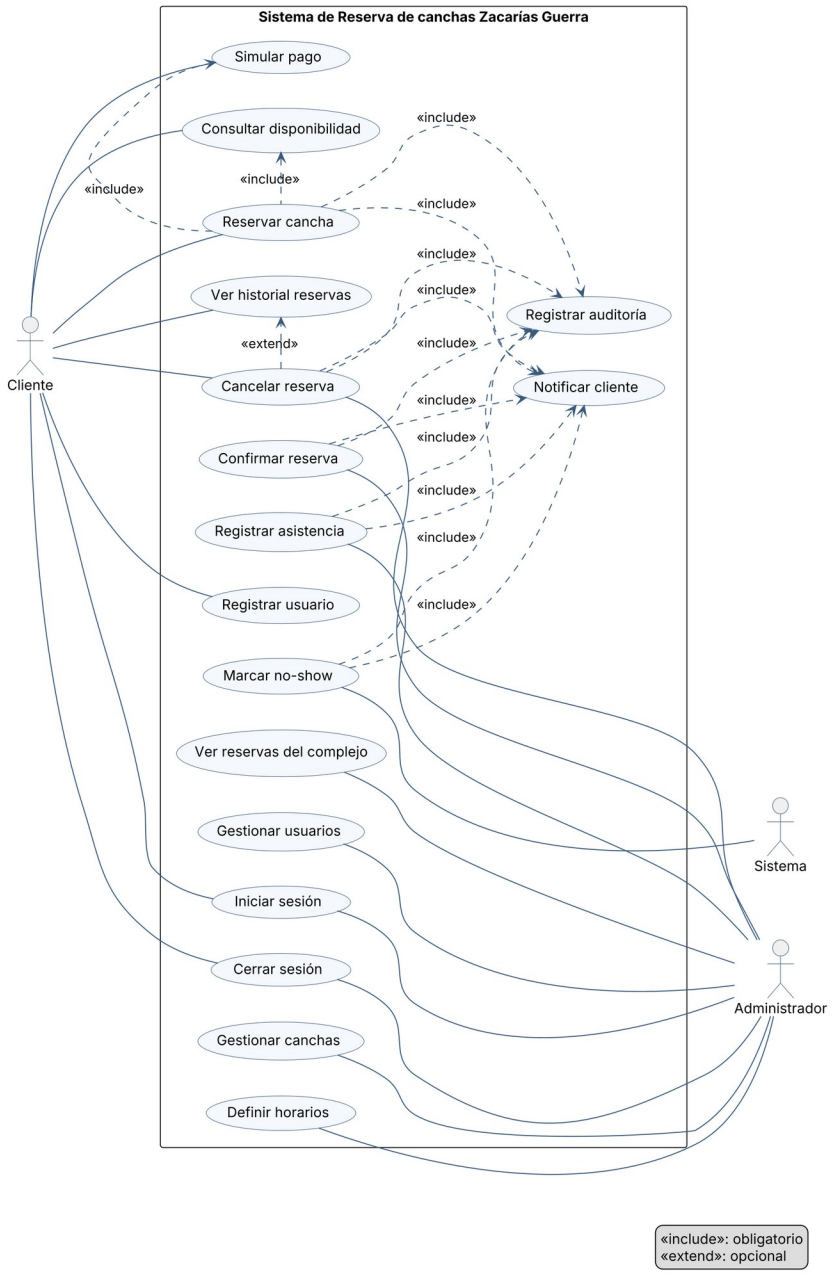


Figura 2. Diagrama general de casos de uso del sistema, corregido

Las Figuras 3 y 4 muestran los casos de uso específicos. En CU-01 el actor se rotula como Cliente y la inclusión de Simular pago va de Reservar cancha hacia Simular pago, como la de Consultar disponibilidad. En CU-02 se corrigió el rótulo, que decía CU-03.

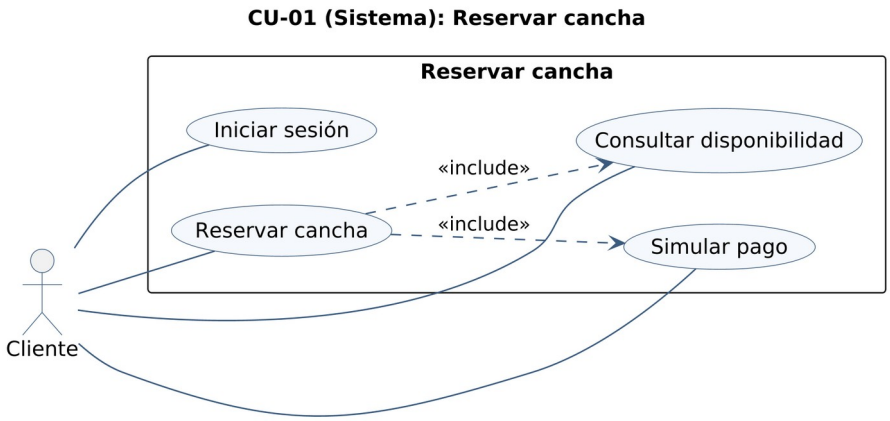


Figura 3. CU-01: Reservar cancha

La ficha de **CU-01, Reservar cancha**, queda así:

- **Actor principal:** Cliente. **Incluye:** Consultar disponibilidad y Simular pago, ambos obligatorios. **Precondiciones:** el Cliente inició sesión y existen canchas dadas de alta.
- **Flujo básico:** el Cliente elige reservar; el sistema muestra los horarios libres por fecha; el Cliente elige cancha, fecha y horario; el sistema ejecuta Simular pago y registra la reserva en estado `PAGADA`; el sistema muestra la confirmación.
- **A1, sin disponibilidad:** si el horario ya no está libre, el sistema ofrece los otros horarios libres de esa cancha ese día, o el Cliente desiste.
- **A2, sesión expirada:** el sistema lleva al Cliente a iniciar sesión y, al autenticarse, lo devuelve al punto donde estaba.
- **Postcondición:** la cancha deja de figurar como disponible en ese horario.

Los cambios respecto de la ficha original son dos: se quitó el punto de extensión "si el Cliente elige pagar en línea", porque el pago forma parte del flujo básico, y el paso 4 registra la reserva en `PAGADA` en lugar de "Reservado". La ficha de CU-02 se conserva sin cambios: el Cliente cancela desde su historial; con 24 horas o más de anticipación no hay penalización y el reembolso es total; con menos, se penaliza el 50 %; el horario se libera, y la cancelación se audita y se notifica. Sus flujos alternativos son el no-show automático y la cancelación por el Administrador en nombre del Cliente.

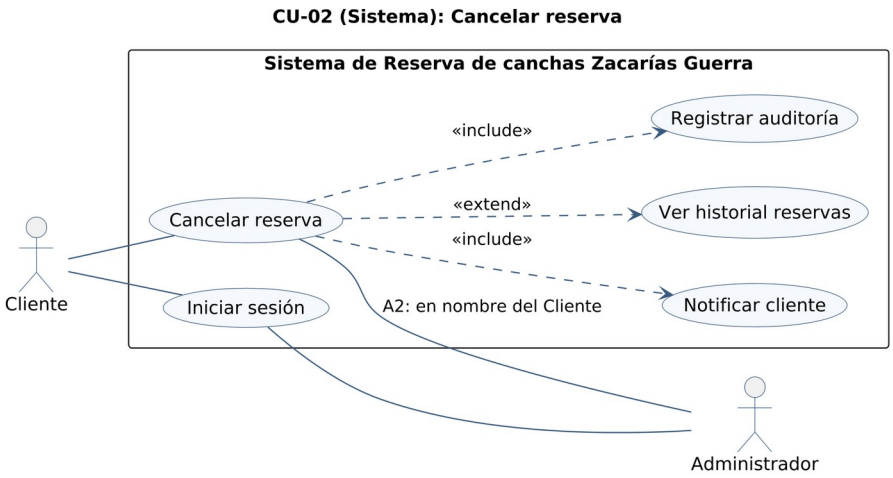


Figura 4. CU-02: Cancelar reserva

1.5 Modelo de contenido

La Figura 5 presenta el diagrama de clases corregido. Se obtuvo directamente del esquema de la base de datos: una clase por tabla, con sus atributos y las restricciones del esquema como enumeraciones. Las clases no llevan operaciones, porque las reglas de negocio viven en la capa de servicio y no en las entidades.

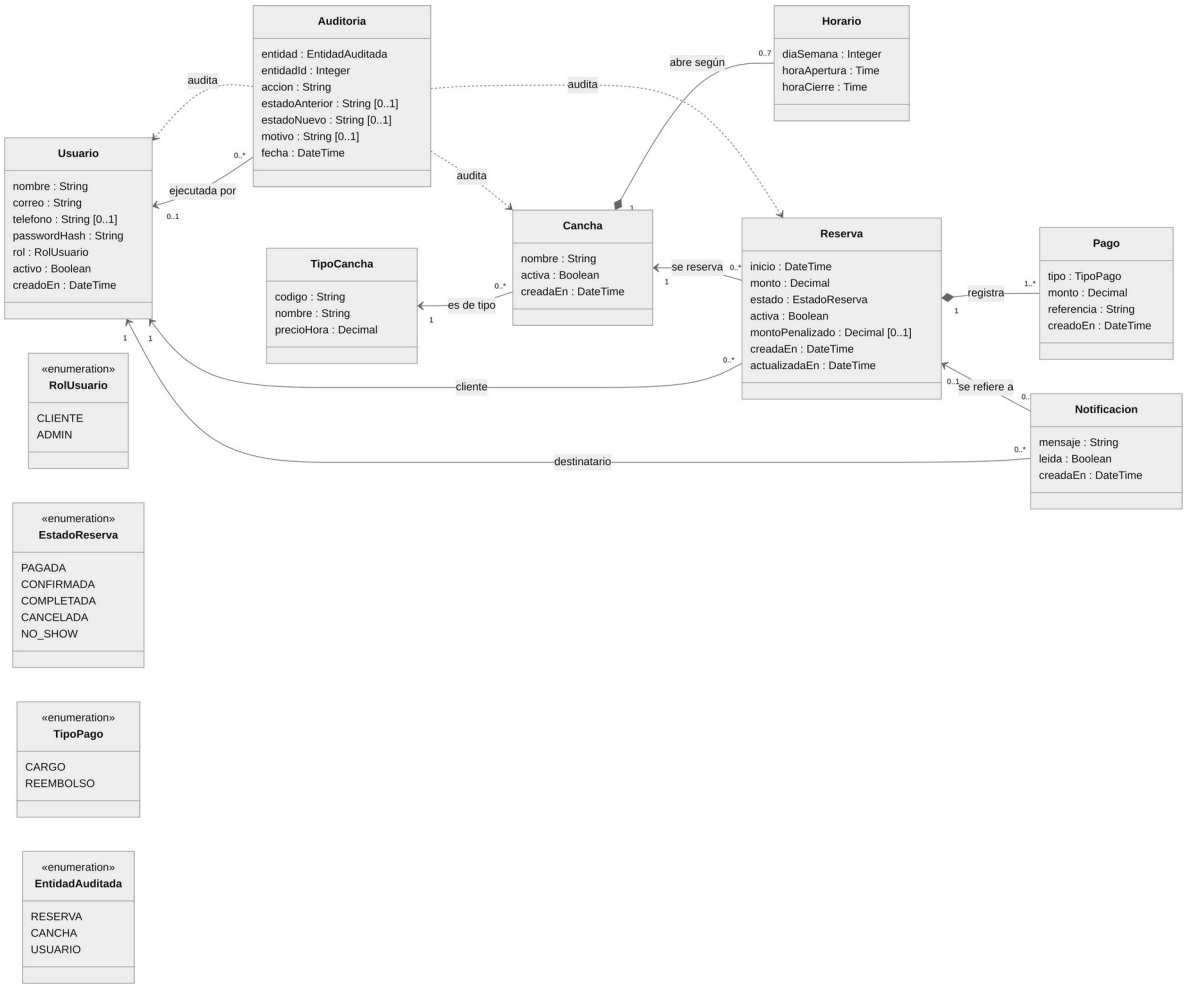


Figura 5. Modelo de contenido (diagrama de clases), corregido

Los cambios respecto del Primer Entregable responden a decisiones tomadas al diseñar el esquema:

- **Usuario** es una sola clase con un atributo `rol`, en lugar de una clase abstracta con Cliente y Administrador como subclases; se implementa en una sola tabla.
- **TipoCancha** es una clase nueva que guarda el código, el nombre y el precio por hora. La Cancha deja de tener `modalidad` y `precioPorHora`, y su `estado` pasa a ser `activa`.
- **Horario** guarda la apertura y el cierre por día de la semana. Los bloques de una hora se calculan; la disponibilidad no se almacena.
- **Reserva** se asocia a una cancha y a una hora de inicio, no a un horario, y guarda el monto copiado del precio vigente y el monto penalizado.
- **Pago, Auditoria y Notificacion** son clases nuevas que exigen el pago simulado, RF13 y RF12.

1.6 Modelo navegacional

La Figura 6 muestra el modelo navegacional. Conserva los 8 nodos y 12 enlaces del Primer Entregable y agrega 5 nodos y 8 enlaces para las pantallas que el sistema terminado necesitó: horario de cancha, reservas del complejo, gestión de usuarios, auditoría y notificaciones, además de la expiración y recuperación de la sesión. Cada nodo corresponde a una página del frontend, y el color indica el rol.

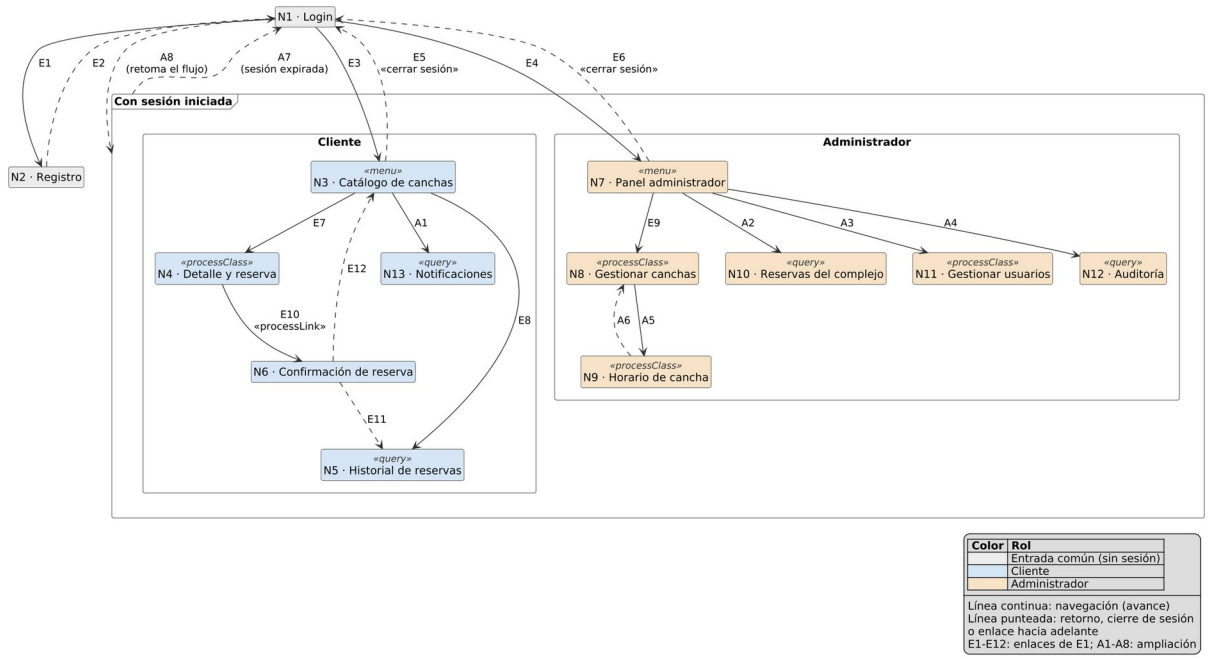


Figura 6. Modelo navegacional (13 nodos y 20 enlaces)

Las acciones con confirmación, como cancelar una reserva, dar de baja una cancha o cambiar el rol de un usuario, se resuelven dentro de la página de su nodo y no tienen nodo propio. El cierre de sesión es una acción del menú persistente de cada rol, disponible en todas sus páginas.

1.7 Modelo de presentación

El Primer Entregable modeló la presentación de dos pantallas: el catálogo de canchas y la confirmación de la reserva (Figuras 7 y 8). Las pantallas implementadas siguen ese diseño en contenido, con dos diferencias: el catálogo no muestra un "estado" por cancha, sino sus bloques libres, porque la disponibilidad se calcula por bloque horario; y los precios del dibujo eran ilustrativos. El estilo visual final es claro, con una hoja de estilos única (sección 4.6).

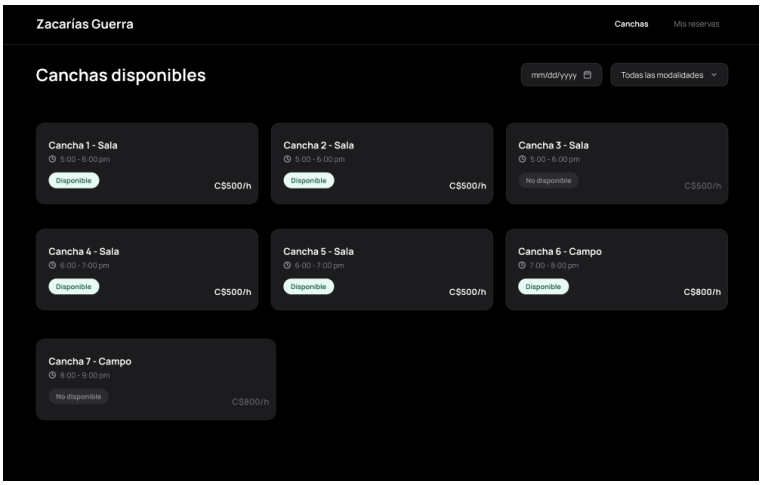


Figura 7. Modelo de presentación del catálogo de canchas (Primer Entregable)

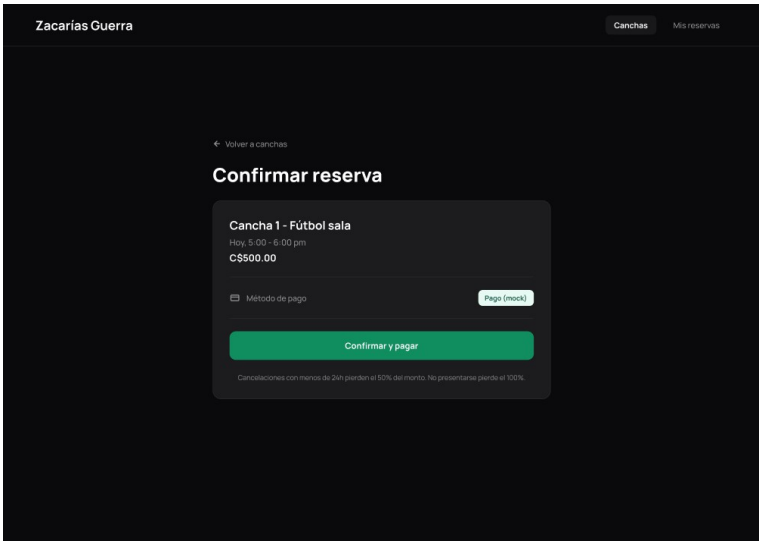


Figura 8. Modelo de presentación de la confirmación de reserva (Primer Entregable)

El modelo de presentación no se completó para las demás pantallas. Por limitación de tiempo, el equipo decidió no hacerlo y registrarlo como compromiso omitido. Las pantallas restantes siguen el modelo navegacional y se validaron con el recorrido completo de la prueba integral (sección 6.2). También quedó omitido el aviso de la regla de penalización al confirmar la reserva: la regla se muestra al cancelar, con el porcentaje y el monto exactos que calcula el servidor.

1.8 Modelo de procesos

La estructura del proceso (Figura 9) y el flujo de reservar (Figura 10) se conservan del Primer Entregable. La estructura distingue las clases de proceso CrearReserva y CancelarReserva y la consulta de disponibilidad. El flujo de reservar coincide con lo implementado: el sistema vuelve a

verificar que el horario siga libre antes de simular el pago, y si se ocupó, devuelve al cliente a los horarios libres.

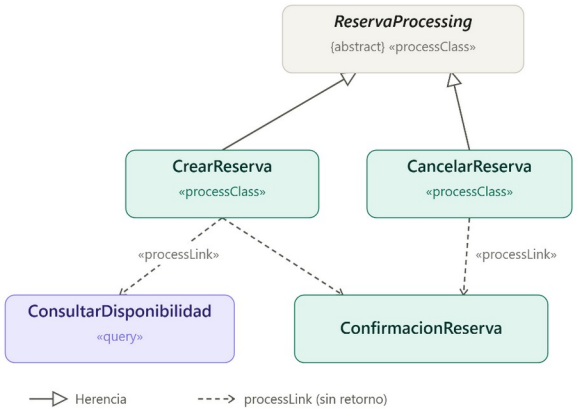


Figura 9. Modelo de estructura del proceso (Primer Entregable)

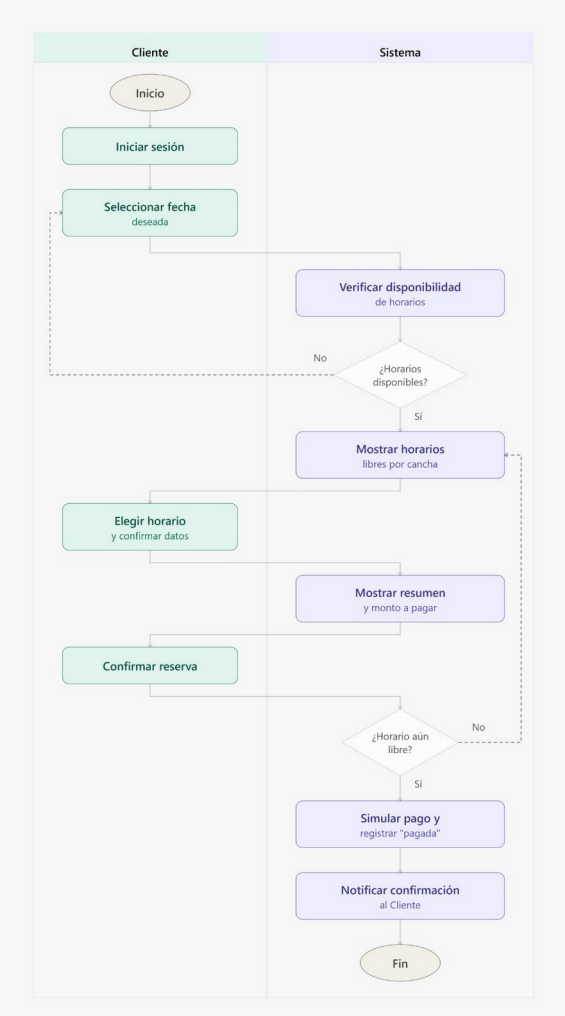


Figura 10. Flujo del proceso de reservar (Primer Entregable)

El Primer Entregable no tenía un diagrama de estados de la reserva, aunque su objetivo específico 3 lo pedía. La Figura 11 lo agrega, derivado de la tabla de transiciones del código. La reserva nace `PAGADA`; el administrador puede confirmarla; desde `PAGADA` o `CONFIRMADA` puede completarse (al registrar la asistencia), cancelarse o marcarse como no-show. Los tres últimos estados son finales. La sección 4.2 detalla la condición y el efecto de cada transición.

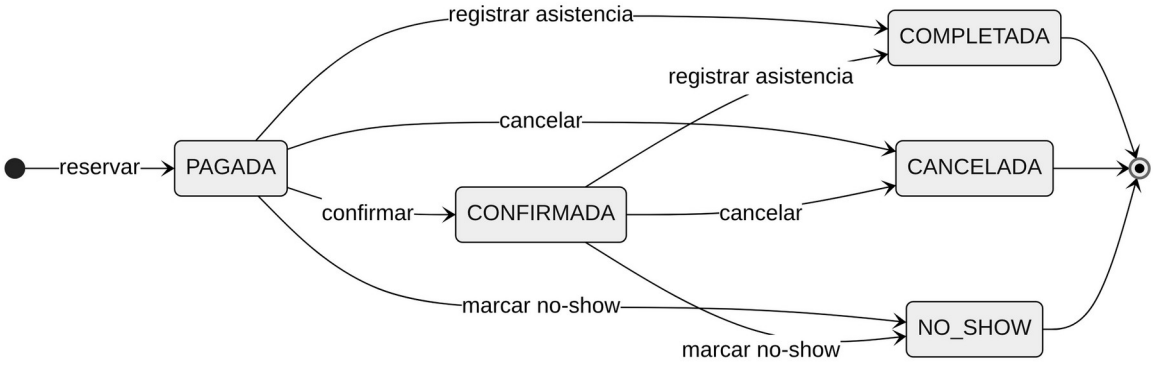


Figura 11. Diagrama de estados de la reserva

La Figura 12 muestra el flujo de baja de cancha corregido. En el original, la rama "No" de "¿Reservas activas?" llevaba a cancelar reservas; ahora lleva directo a actualizar el estado de la cancha, que es lo que la lógica del flujo exige.

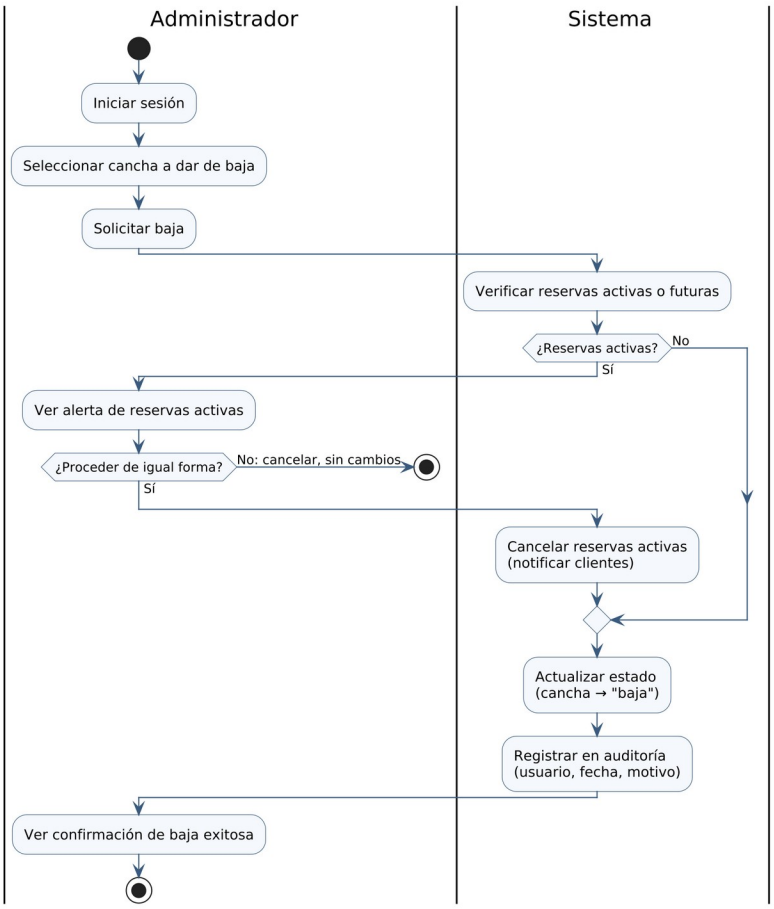


Figura 12. Flujo del proceso de baja de cancha, corregido

1.9 Cambios respecto del Primer Entregable

Tabla 3. Resumen de cambios en los modelos UWE

Modelo	Cambio	Motivo
Casos de uso general	Simular pago pasa de extensión a inclusión de Reservar cancha	El pago simulado siempre ocurre
Casos de uso general	Registrar usuario deja de asociarse al Administrador	El registro público crea solo clientes
Casos de uso general	Se agregan Registrar asistencia, Marcar no-show (actor Sistema) y Registrar auditoría	El sistema implementa esas funciones
CU-01	Sin punto de extensión de pago; la reserva nace PAGADA; actor y sentido de la inclusión corregidos	El pago es parte del flujo básico
CU-02	Rótulo corregido (decía CU-03)	Error de rotulado
Contenido	Usuario en una sola clase; TipoCancha, Pago, Auditoria y Notificacion nuevas; Horario por día de la semana; sin operaciones	Diseño del esquema y reglas en la capa de servicio
Procesos	Diagrama de estados nuevo	Lo pedía el objetivo específico 3
Procesos	Flujo de baja: la rama "No" va a actualizar el estado	Error de flujo
Navegacional	5 nodos y 8 enlaces nuevos; diagrama redibujado	Pantallas que el sistema necesitó
Presentación	No se completó para las demás pantallas	Tiempo; compromiso omitido

Capítulo II: Arquitecturas de software para la web moderna

2.1 Arquitectura seleccionada

El Segundo Entregable adoptó el modelo cliente-servidor: el navegador realiza solicitudes HTTP a una API centralizada que concentra la lógica de negocio y el acceso a los datos. Se descartó un modelo entre pares porque toda operación depende de un único punto de verdad sobre la disponibilidad de las canchas. Para el servidor se seleccionó una arquitectura monolítica, por cuatro razones: la complejidad operativa es proporcional al alcance del sistema; la regla más sensible, impedir dos reservas en el mismo horario (RNF09), se resuelve con una transacción ACID sobre una única base de datos; el equipo tenía experiencia en arquitecturas de capas y no en sistemas distribuidos; y el hosting gratuito disponible está orientado a desplegar un único servicio.

La implementación confirmó la elección. El sistema terminado es un solo proceso, construido como una sola imagen Docker y desplegado como un único servicio.

2.2 Arquitectura en capas, como se implementó

La Figura 13 muestra cómo quedó organizado el sistema. Dentro del monolito, la arquitectura en capas se aplicó por módulo funcional: cada módulo del servidor tiene sus propias rutas, controlador, servicio y repositorio, y un módulo solo se comunica con otro a través de su servicio.

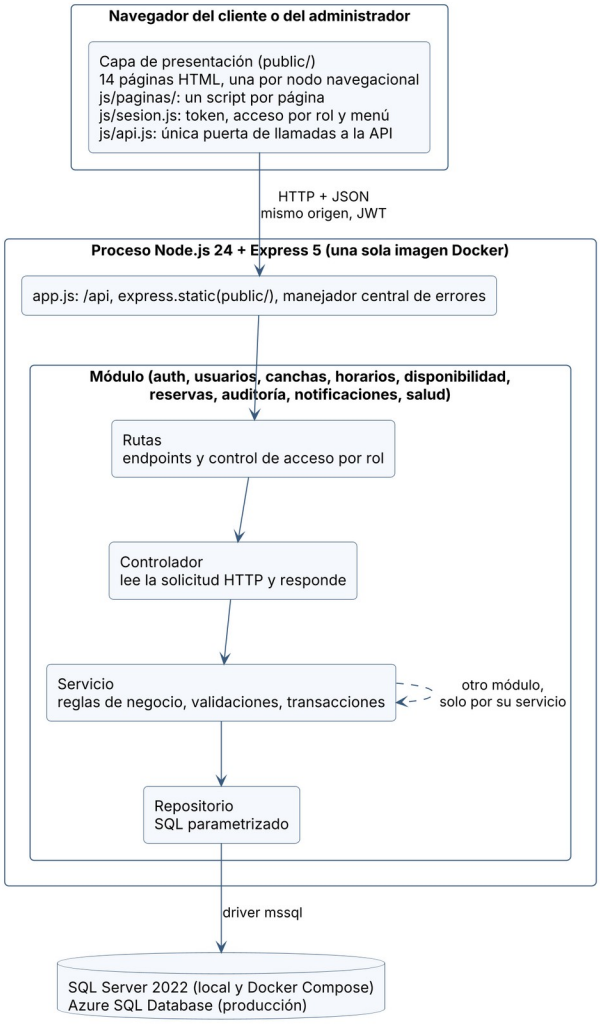


Figura 13. Arquitectura en capas del sistema implementado

La implementación se apartó del Segundo Entregable en dos puntos:

- **Sin ORM:** el acceso a datos usa SQL escrito a mano con parámetros, sobre el driver `mssql`. Así se expresan directamente en el SQL los bloqueos que exige el control de concurrencia (sección 4.4).
- **Frontend no independiente:** la capa de presentación consume la API por HTTP, pero el mismo servidor Express la entrega como archivos estáticos. API y pantallas comparten origen, imagen y despliegue, lo que simplifica la seguridad (sin peticiones entre orígenes) y mantiene un único servicio.

El dominio también creció respecto de las seis entidades del Segundo Entregable: Usuario quedó en una sola tabla, y se agregaron TipoCancha, Pago, Notificacion y Auditoria (sección 4.1).

2.3 Ventajas y limitaciones observadas

Las ventajas anticipadas se verificaron. La separación por módulo y por capa permitió que los tres integrantes trabajaran en paralelo en carpetas distintas, con pocos conflictos al integrar. La lógica de negocio se probó sin base de datos, sustituyendo el repositorio por dobles de prueba: las 710 pruebas automatizadas corren en segundos y en cada pull request. Un único repositorio y una única imagen simplificaron la integración y el despliegue continuos.

Las limitaciones también se confirmaron, sin afectar el alcance. Un fallo no controlado afecta a todo el sistema, por lo que se centralizó el manejo de errores en un solo componente. El escalamiento es solo vertical, aceptable para un complejo de siete canchas: la consulta de disponibilidad respondió en 544 ms en el peor caso medido en producción (sección 6.3). Y la disciplina entre capas tuvo que sostenerse con revisión de código, porque nada impide técnicamente que una capa salte a otra.

2.4 Posible migración

El Segundo Entregable planteó una migración gradual si el complejo creciera: primero el monolito actual; después, la extracción de Reservas como servicio, por ser el de mayor carga transaccional; luego Notificaciones, por ser naturalmente asíncrono; y finalmente la gestión de canchas, solo si se administraran varios complejos. La organización por módulos facilita ese camino, porque cada módulo ya expone sus funciones a los demás a través de su servicio. Mientras no se justifique separar los datos, se mantendría una única base como fuente de verdad.

Capítulo III: Análisis de la tecnología y del desarrollo

3.1 Tecnología utilizada

El sistema terminado conserva la tecnología del backend descrita en el Tercer Entregable y le agrega el frontend, la contenedorización y la integración y el despliegue continuos. Todos los componentes son gratuitos, de modo que el proyecto se desarrolla y se despliega sin costo.

Tabla 4. Tecnología utilizada

Componente	Elección	Justificación
Entorno de ejecución	Node.js 24 LTS	Versión con soporte de largo plazo, con ejecutor de pruebas nativo.
Framework HTTP	Express 5	Propaga los errores de los manejadores asíncronos a un manejador central. También entrega el frontend como archivos estáticos.
Base de datos	SQL Server Express en desarrollo; Azure SQL Database en producción	Transacciones ACID e índices únicos filtrados, que impiden dos reservas activas en la misma cancha y hora (RNF09). Azure SQL es el mismo motor en la nube, con una oferta gratuita que pausa la base cuando no se usa.
Acceso a datos	Driver mssql, SQL parametrizado y migraciones versionadas	Protege contra la inyección SQL y da control explícito de los bloqueos en las transacciones críticas.
Contraseñas	bcryptjs	Hash lento con sal (RNF02).
Sesión	JSON Web Tokens en sessionStorage	Autenticación sin estado en el servidor y control de acceso por rol (RNF03, RF14). El token se borra al cerrar la pestaña o la sesión.
Frontend	HTML, CSS y JavaScript sin framework, con módulos nativos	Sin paso de compilación. Una página por nodo navegacional y una hoja de estilos única, usable desde el celular (RNF05).
Pruebas	Ejecutor nativo de Node (node --test)	Pruebas automatizadas del backend y del frontend que no necesitan base de datos y corren en cada pull request.
Contenedores	Docker y Docker Compose	Una sola imagen para desarrollo, CI y producción. Compose levanta la aplicación y SQL Server 2022 como entorno reproducible.
CI/CD	GitHub Actions y GitHub Container Registry	Pruebas, construcción de la imagen con prueba de humo y control de autoría en cada cambio; publicación y despliegue al integrar en main.

Componente	Elección	Justificación
Alojamiento	Render, plan gratuito	Ejecuta la imagen como un proceso permanente, que es lo que la aplicación requiere. Se descartó una plataforma de funciones sin servidor (Vercel), porque ejecuta Express como una función y no entrega el frontend estático.
Configuración	Variables de entorno	Las credenciales nunca se guardan en el repositorio. En producción viven en Render y en los secretos de GitHub.

3.2 Patrón de diseño

El backend aplica la arquitectura en capas por módulo funcional: autenticación, usuarios, canchas (con los tipos de cancha), horarios, disponibilidad, reservas, auditoría, notificaciones y salud. Cada módulo contiene sus propias capas, con responsabilidades fijas (Tabla 5).

Tabla 5. Capas de cada módulo del backend

Capa	Responsabilidad	No debe
Rutas	Declarar los endpoints y los controles de acceso por rol	Contener lógica
Controlador	Leer la solicitud HTTP, invocar al servicio y responder	Contener SQL ni reglas de negocio
Servicio	Reglas de negocio, validaciones y transacciones	Conocer la solicitud HTTP ni escribir SQL
Repositorio	Consultas SQL parametrizadas	Decidir si una operación está permitida

Cada capa solo invoca a la inmediatamente inferior, y un módulo solo usa a otro a través de su servicio. Las funciones que un módulo ofrece a los demás tienen firmas fijas, acordadas antes de implementarse y documentadas en el repositorio; así, por ejemplo, la consulta de disponibilidad no tiene repositorio propio y combina los servicios de canchas, horarios y reservas. Tres decisiones complementan el patrón:

- **Reglas puras:** el cálculo de la penalización y la tabla de transiciones de la reserva están en un archivo de funciones sin base de datos ni reloj; el "ahora" llega como parámetro. Eso permite probar sus bordes exactos, como la cancelación con exactamente 24 horas de anticipación.

- **Errores uniformes:** los servicios lanzan errores de aplicación con un estado HTTP y un código (por ejemplo, `HORARIO_OCUPADO` o `TRANSICION_INVALIDA`), y un manejador central los convierte en una respuesta con la misma forma para toda la API.
- **Frontend en tres piezas:** un módulo concentra todas las llamadas a la API, otro la sesión, el acceso por rol y el menú, y cada página tiene un único script. Ninguna página llama a la API directamente.

El patrón cumplió su propósito: cada integrante trabajó en su módulo con conflictos mínimos, las reglas críticas quedaron aisladas y probadas, y un cambio en el SQL de un módulo no afectó a los demás. Sus limitaciones son las del monolito, ya tratadas en la sección 2.3.

3.3 Estrategias de desarrollo

El proyecto se desarrolló con Git y GitHub, con un flujo basado en issues, ramas por tarea, pull requests con revisión entre pares e integración y despliegue continuos (Capítulo V).

- **Issues:** cada tarea se registró como un issue, etiquetado con el rol responsable, y su especificación quedó en el cuerpo del issue. Los errores se reportaron también como issues, con la etiqueta `bug`. El plan del sprint final y sus decisiones se centralizaron en un issue fijado (#35).
- **Ramas:** `main` contiene solo versiones entregadas y es lo que está en producción; `develop` integra el trabajo terminado. Cada tarea se desarrolló en su propia rama, con un prefijo según el tipo de cambio (`feature/`, `fix/`, `docs/`, `chore/`, `ci/`, `test/`). Nadie trabajó directamente sobre `main` ni `develop`.
- **Pull requests:** todo cambio entró por pull request, con una plantilla que pide qué cambia, qué requerimientos cubre, cómo se probó y qué queda pendiente. Las reglas de protección de `develop` y `main` exigen la aprobación de otro integrante y las tres comprobaciones de integración continua en verde.
- **Revisión:** la revisa el primero disponible de los otros dos integrantes, con mayor profundidad en los módulos críticos (disponibilidad, creación y cancelación de reservas, y baja de cancha). La rotación fija prevista en el Tercer Entregable se reemplazó por esta regla, que evitó cuellos de botella.
- **Commits:** los mensajes siguen un formato de tipo y alcance, como `feat(reservas): …` o `docs(guion): …`, lo que vuelve el historial legible y trazable.
- **Decisiones de contrato:** todo cambio que afecta a otros módulos (firmas, columnas, estados, endpoints o reglas de negocio) se registró como comentario en el issue del plan antes de implementarse.

3.4 Equipo y distribución del trabajo

El equipo lo formaron tres integrantes, con un rol cada uno. La Tabla 6 resume los frentes de cada rol al cierre del proyecto.

Tabla 6. Distribución del trabajo por rol

Rol	Frentes principales
A: Matthew Muñoz (coordinación)	Estructura del proyecto, base de datos, autenticación, canchas, horarios, disponibilidad y baja de cancha; Docker, integración y despliegue continuos y producción; pantallas de administración de canchas, horarios y reservas; modelos de contenido y de estados.
B: Álvaro Garay	Usuarios, consultas de reservas y prueba de concurrencia; guion de demostración y sus corridas, medición de RNF04, matriz de trazabilidad, modelos UWE corregidos y documentación del despliegue.
C: Keneth Palacios	Reservas (creación con pago simulado, estados, cancelación y no-show), auditoría y notificaciones; frontend base y pantallas de cliente, de usuarios y de auditoría.

El reparto se ajustó varias veces, siempre con registro en el repositorio. El 21/10, por la disponibilidad reducida de un integrante, autenticación pasó al rol A y auditoría y notificaciones al rol C (#31). En el sprint final, la regla del plan permitía reasignar las tareas de un hito atrasado: las consultas de reservas pasaron al rol B (02/11), el frontend base al rol C (03/11) y las pantallas de usuarios y auditoría al rol C (05/11). El 11/11, los modelos de contenido y de estados pasaron al rol A.

Al 13/11/2026, el repositorio registraba 42 issues, 51 pull requests y 228 commits en `develop`, en un período de cinco semanas desde el primer commit, el 11/10/2026.

3.5 Repositorio y versiones

- **Repositorio:** https://github.com/Mattmnz/reservas-zacarias-guerra (público).
- **Producción:** https://reservas-zacarias-guerra.onrender.com
- **Documentación en el repositorio:** el README describe el sistema, cómo ejecutarlo y cómo se trabaja; la carpeta `docs/` contiene la matriz de trazabilidad, los modelos UWE, el guion de demostración con sus corridas, el despliegue y las peticiones de prueba por módulo.

Se publicaron tres versiones, cada una como etiqueta sobre `main` (Tabla 7). La defensa prevista para el 07/11 se trasladó y la entrega final quedó para el 14/11; ese tiempo se usó para cerrar la documentación, que se publicó como `v1.0.1` sin cambios en el código.

Tabla 7. Versiones publicadas

Versión	Fecha	Contenido
v0.3-backend	05/11/2026	Backend completo y primer despliegue en producción (versión preliminar).
v1.0	06/11/2026	Entrega final del sistema: todas las pantallas, integración y despliegue continuos desde la imagen publicada, guion de demostración y matriz de trazabilidad.
v1.0.1	13/11/2026	Versión de documentación: modelos UWE corregidos, matriz de trazabilidad sin compromisos pendientes, registro de la corrida complementaria y este informe. El código es el mismo de v1.0.

Capítulo IV: Implementación del sistema

4.1 Modelo de datos

La base de datos tiene ocho tablas, creadas por migraciones SQL versionadas en el repositorio (Tabla 8). Varias reglas de negocio se garantizan en el propio esquema, de modo que ni un error de la aplicación pueda violarlas.

Tabla 8. Tablas de la base de datos

Tabla	Contenido	Restricciones relevantes
Usuario	Clientes y administradores	Correo único y en minúsculas; rol CLIENTE o ADMIN; activo
TipoCancha	Modalidad y precio por hora	Código único; precio mayor que cero
Cancha	Las canchas del complejo	Nombre único; baja lógica con el campo Activa
Horario	Apertura y cierre por día de la semana	Uno por cancha y día; horas en punto; cierre posterior a la apertura
Reserva	Bloque de una hora reservado	Estado dentro de los cinco válidos; Activa falso solo si está cancelada; inicio en hora en punto; monto penalizado entre cero y el monto
Pago	Cargos y reembolsos simulados	Tipo CARGO o REEMBOLSO; referencia única; monto mayor que cero
Auditoria	Cambios de reservas, canchas y usuarios	Entidad RESERVA, CANCHA o USUARIO; usuario nulo para las acciones del sistema
Notificacion	Avisos para los usuarios	Marca de leída; referencia opcional a la reserva

El precio pertenece al tipo de cancha y se copia en la reserva al crearla, de modo que un cambio de precio posterior no altera las reservas existentes. Con los datos iniciales, la hora de fútbol sala cuesta C$ 600 y la de fútbol campo C$ 1,200, y todas las canchas abren de 07:00 a 22:00. Agregar una cancha o una modalidad es insertar una fila, sin cambios de esquema (RNF08). Las fechas se manejan siempre en hora de Nicaragua, calculada por la aplicación y no por el servidor de base de datos, para que las reglas de anticipación den el mismo resultado en desarrollo y en producción.

4.2 Reglas de negocio

La Tabla 9 resume las reglas que el sistema aplica. Las de transición de estado y penalización están implementadas como funciones puras con pruebas de sus bordes (sección 3.2).

Tabla 9. Reglas de negocio implementadas

Regla	Implementación
Bloques horarios	Las reservas son de una hora, en punto, dentro del horario de la cancha ese día.
Anticipación	Se reserva un bloque futuro hasta 30 días después del momento actual. La disponibilidad aplica el mismo filtro, así que nunca ofrece un bloque que la creación rechazaría.
Pago simulado	Al reservar se registra un cargo por el monto, con referencia SIM-C-<id>, en la misma transacción que la reserva. La reserva nace PAGADA.
Confirmación	El administrador confirma una reserva PAGADA que todavía no empezó.
Cancelación	El cliente dueño o el administrador cancela una reserva que no empezó. Con 24 horas o más de anticipación, penalización de 0 % y reembolso total; con menos, penalización de 50 % y reembolso del resto, con referencia SIM-R-<id>. Antes de confirmar, la pantalla muestra el porcentaje y los montos que calcula el servidor.
Asistencia	El administrador la registra desde 15 minutos antes del inicio hasta el fin del bloque. La reserva pasa a COMPLETADA, aunque no se haya confirmado.
No-show	Una reserva PAGADA o CONFIRMADA cuyo inicio pasó hace una hora o más se marca NO_SHOW, con penalización de 100 % y sin reembolso. No hay tarea programada: el sistema marca las reservas vencidas antes de cada operación que lee o cambia reservas, y antes de consultar la disponibilidad.
Baja de cancha	Es lógica. Si la cancha tiene reservas activas futuras o en curso, el sistema informa cuántas y exige una segunda confirmación; al confirmar, las cancela con reembolso total.
Auditoría y notificación	Cada cambio de estado se audita y se notifica al cliente en la misma transacción que el cambio.

4.3 API REST

La API expone 28 endpoints bajo la ruta `/api`, agrupados por módulo (Tabla 10). Todos, salvo el registro, el inicio de sesión y el chequeo de salud, exigen un token válido; las operaciones de administración exigen además el rol de administrador. Las respuestas tienen una forma común: los datos bajo `data`, o un error con su código y mensaje.

Tabla 10. Endpoints de la API por módulo

Módulo	Endpoints	Acceso
Autenticación	POST /auth/registro, POST /auth/login, GET /auth/yo	Público, salvo /yo
Usuarios	GET /usuarios, PATCH /usuarios/:id	Administrador
Canchas	GET /canchas, GET /canchas/:id; POST, PUT /canchas/:id, PATCH /canchas/:id/alta y /baja	Consulta autenticada; cambios del administrador
Tipos de cancha	GET /tipos-cancha; PUT /tipos-cancha/:id/precio	Consulta autenticada; precio del administrador
Horarios	GET y PUT /canchas/:id/horarios	Consulta autenticada; cambio del administrador
Disponibilidad	GET /disponibilidad?fecha=&tipo=	Autenticado
Reservas	POST /reservas; GET /reservas/mias; GET /reservas y /reservas/:id; PATCH /:id/confirmar y /:id/asistencia; GET /:id/cancelacion; POST /:id/cancelar	Crear y listar las propias: cliente. Listar todas, confirmar y asistencia: administrador. Detalle y cancelación: dueño o administrador
Notificaciones	GET /notificaciones, PATCH /notificaciones/:id/leida	Autenticado, sobre las propias
Auditoría	GET /auditoria?entidad=&entidadId=	Administrador
Salud	GET /health (ejecuta una consulta a la base)	Público

4.4 Control de concurrencia

La regla más sensible del sistema es que dos reservas simultáneas no ocupen el mismo horario de la misma cancha (RNF09). Se resolvió con una doble defensa dentro de la transacción que crea la reserva:

1. La transacción lee el bloque con un bloqueo de rango (`UPDLOCK, HOLDLOCK`). Una segunda solicitud simultánea sobre el mismo bloque espera a que la primera termine, en lugar de leerlo como libre.
2. Un índice único filtrado sobre cancha e inicio, limitado a las reservas activas, garantiza en el motor que no existan dos reservas activas para el mismo bloque. Si la primera defensa fallara, la violación del índice se traduce en el mismo error `HORARIO_OCUPADO` que recibe el cliente.

El índice filtra por reservas activas porque una cancelación debe liberar el bloque: las canceladas quedan con `Activa = 0` y dejan de contar. Las demás reservas terminadas (completadas o no-show) siguen activas y conservan el bloque ocupado. La sección 6.4 presenta la prueba con diez solicitudes simultáneas.

4.5 Seguridad

- **Contraseñas:** se guardan con hash bcrypt (costo 10) y se exige un mínimo de 8 caracteres (RNF02).
- **Autenticación y roles:** el inicio de sesión entrega un token JWT firmado. Cada solicitud protegida verifica el token, que el usuario siga activo y su rol (RNF03, RF14). El registro público solo crea clientes.
- **Frontend:** el token se guarda en `sessionStorage`, que se borra al cerrar la pestaña. Los datos del servidor se insertan siempre como texto, nunca como HTML, lo que evita la inyección de código. El regreso tras iniciar sesión solo acepta rutas del mismo sitio.
- **Base de datos:** todo el SQL es parametrizado. En producción la aplicación se conecta con un usuario que solo puede leer y escribir filas; las migraciones se ejecutan aparte, con el administrador del servidor.
- **Credenciales:** ninguna se guarda en el repositorio; se configuran como variables de entorno y secretos.

4.6 Interfaz web

El frontend tiene una página por nodo del modelo navegacional, con un menú persistente por rol. Toda pantalla muestra los estados de carga, de lista vacía y de error, pide confirmación antes de las acciones con efecto y muestra explícitos los montos penalizados y reembolsados. Una hoja de estilos única organiza el contenido en una columna en el celular y en dos desde tabletas, con áreas táctiles de 44 píxeles (RNF05). Las Figuras 14 a 19 son capturas del sistema en producción.

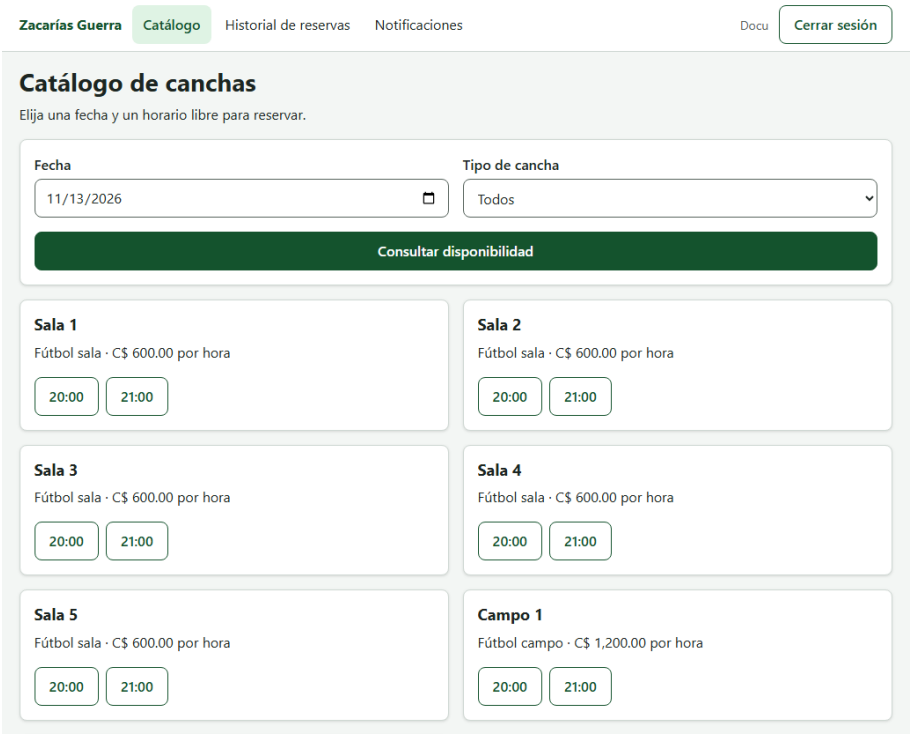


Figura 14. Catálogo de canchas: disponibilidad por fecha y tipo, con los bloques libres de cada cancha



Figura 15. Detalle y reserva: resumen con el monto y aviso de pago simulado



Figura 16. Historial de reservas del cliente, con el estado, los pagos y la opción de cancelar

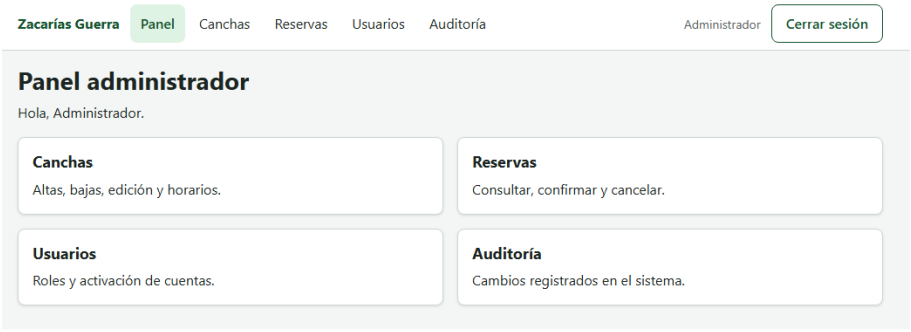


Figura 17. Panel del administrador

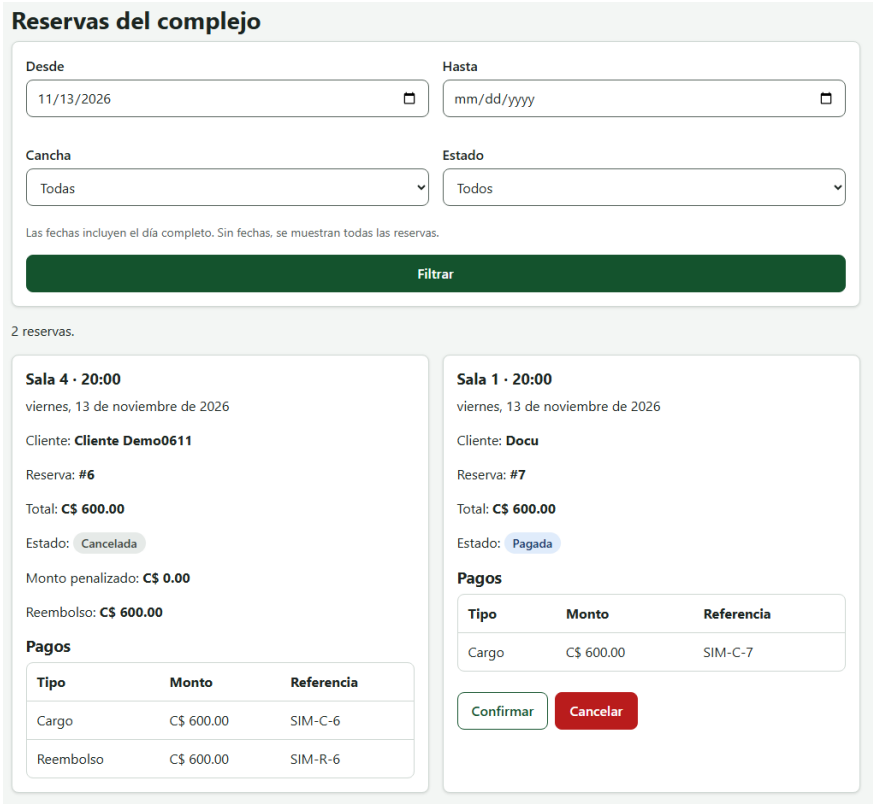


Figura 18. Reservas del complejo: filtros, estado, pagos, reembolso y acciones del administrador

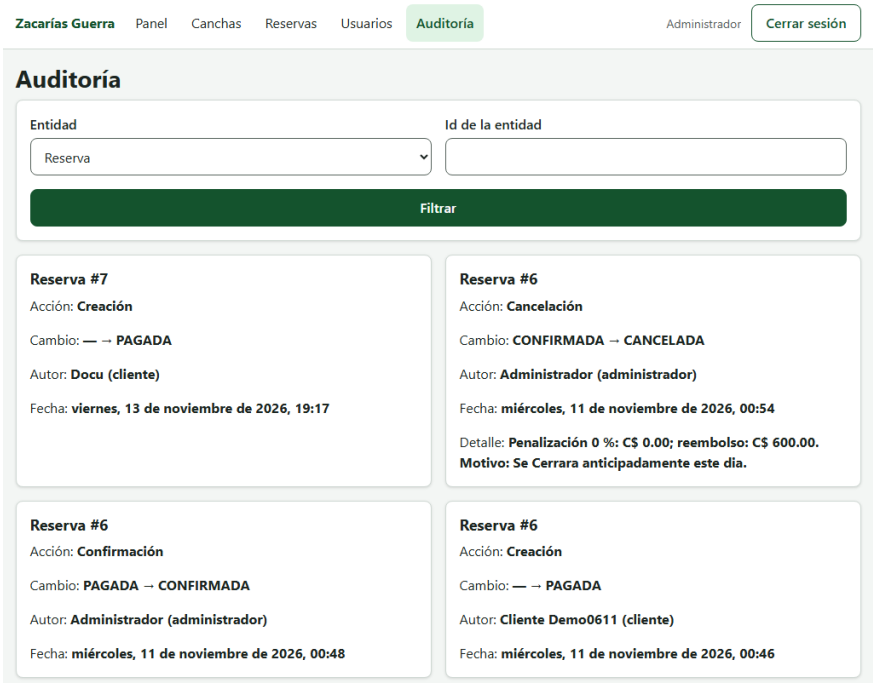


Figura 19. Auditoría: acción, cambio de estado, autor, fecha y detalle de cada registro

4.7 Auditoría y notificaciones

La auditoría registra cada cambio con la entidad y su identificador, la acción, el estado anterior y el nuevo, el usuario responsable (o el sistema, en el no-show), la fecha y un motivo. Cubre las reservas (creación, confirmación, asistencia, cancelación y no-show, con la penalización y el reembolso en el motivo), las canchas (creación, edición, baja, reactivación y cambio de horario) y los usuarios (cambio de rol, activación y desactivación) (RF13). El administrador la consulta filtrando por entidad e identificador (Figura 19).

Cada cambio de estado de una reserva genera una notificación para el cliente, en la misma transacción que el cambio (RF12). El menú del cliente muestra la cantidad de notificaciones sin leer, y el cliente las marca como leídas desde su pantalla.

Capítulo V: Contenedores, integración y despliegue continuos

5.1 Docker y Docker Compose

La aplicación se construye como una sola imagen Docker, sobre `node:24-alpine`, que se usa sin cambios en desarrollo, en la integración continua y en producción. La imagen instala solo las dependencias de producción, corre con un usuario sin privilegios y recibe la señal de detención directamente, de modo que el servidor cierra la conexión a la base antes de salir. Los archivos de entorno, las pruebas y la documentación quedan fuera de la imagen.

Docker Compose levanta el entorno completo con un solo comando: SQL Server 2022 en una versión fijada, un servicio que crea la base, aplica las migraciones y carga los datos iniciales, y la aplicación, que arranca solo cuando la base está lista. Sirve como entorno reproducible para cada integrante y como respaldo de la demostración si el entorno público fallara.

5.2 Integración continua

Cada pull request, y cada push a `develop` y a `main`, ejecuta tres comprobaciones en GitHub Actions:

- **Pruebas:** instala las dependencias y ejecuta las 710 pruebas automatizadas.
- **Imagen Docker:** valida el archivo de Compose, construye la imagen y le corre una prueba de humo: levanta el contenedor sin base de datos y comprueba que el frontend responde, que la API rechaza una solicitud sin token y que una ruta inexistente da 404.
- **Autoría de commits:** rechaza los commits cuyos mensajes declaran coautoría o generación automática.

Las reglas de protección de `develop` y `main` exigen las tres comprobaciones en verde y la aprobación de otro integrante antes de integrar.

5.3 Despliegue continuo y producción

La Figura 20 resume el recorrido de un cambio hasta producción. Al integrarse en `main`, y solo con las tres comprobaciones en verde, un job construye la imagen, le corre la prueba de humo y sube esa misma imagen a GitHub Container Registry con dos etiquetas: `latest` y la del commit. Después, otro job dispara el despliegue en Render mediante un deploy hook guardado como secreto, y Render descarga la imagen `latest`. Así, lo que corre en producción es exactamente la imagen que pasó las pruebas.

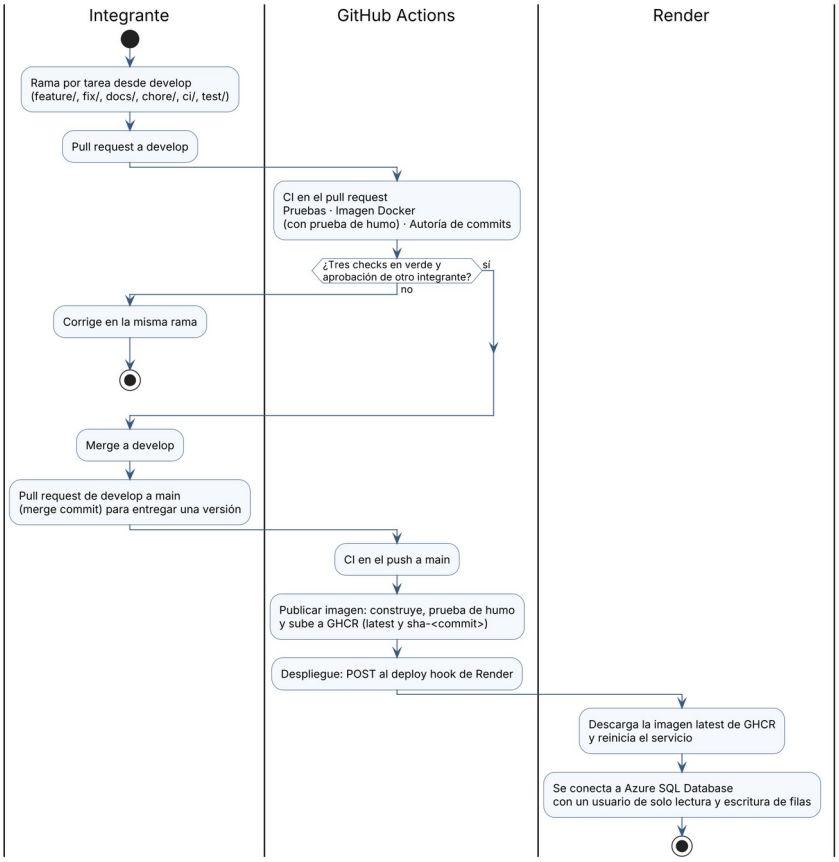


Figura 20. Recorrido de un cambio: integración y despliegue continuos

La producción se compone de Render, que ejecuta la imagen como servicio web en el plan gratuito, y Azure SQL Database, en su oferta gratuita, porque el hosting gratuito no ofrece disco persistente ni la memoria que SQL Server requiere. Ambos se pausan cuando nadie los usa: el servicio tarda cerca de un minuto en volver, y por eso se despierta antes de cada demostración. La base se configuró para pausarse al agotar el cupo gratuito mensual, en lugar de generar cargos.

Para volver a una versión anterior se usa el rollback de Render, o se apunta el servicio a la etiqueta del commit de una versión previa; la corrección entra después por el mismo flujo. El procedimiento está documentado, pero no se probó en producción. Los pasos de creación, configuración y operación del entorno están en el documento de despliegue del repositorio.

5.4 Relación con la guía del docente

El proyecto tomó como referencia la guía de Docker y CI/CD compartida por el docente y cubre sus piezas: un Dockerfile con la imagen base fijada y su `.dockerignore`; integración continua en cada pull request y en cada push, con prueba de humo del contenedor; publicación de la imagen en GitHub Container Registry con las etiquetas `latest` y del commit; despliegue continuo en Render

por deploy hook, solo desde `main` y después de las comprobaciones; y rollback desde el panel de Render o por la etiqueta de la imagen.

Se aparta de la guía en un detalle: la guía vuelve a construir la imagen al publicarla, mientras que el proyecto sube la misma imagen que pasó la prueba de humo. Además, la integración continua del proyecto agrega las pruebas automatizadas, la validación de Docker Compose y el control de autoría.

Capítulo VI: Pruebas y validación

6.1 Pruebas automatizadas

El proyecto tiene 710 pruebas automatizadas en 25 archivos: 578 del backend y 132 de los módulos del frontend. Cubren las reglas de negocio con sus bordes (la cancelación con exactamente 24 horas, el bloque que empieza justo ahora, la reserva a exactamente 30 días), los servicios con el repositorio sustituido por dobles de prueba, los controladores, el control de acceso por rol y los módulos de formato, sesión y parámetros del frontend. Corren sin base de datos, en unos segundos, en cada pull request.

Su límite es explícito: como sustituyen el acceso a datos, no ejercitan el SQL real. Las consultas se verificaron con las peticiones de prueba de cada módulo contra una base local, con la prueba de concurrencia (sección 6.4) y con la prueba integral contra producción (sección 6.2).

6.2 Prueba integral contra producción

El guion de demostración recorre el sistema completo por pantalla, con un cliente y un administrador, en 39 pasos. Cubre el registro y el inicio de sesión, el control de acceso por rol, la reserva con pago simulado y sus alternativas, la cancelación con y sin penalización, la confirmación, la asistencia, el no-show, la gestión de canchas, horarios y usuarios, la auditoría, la vista móvil y la evidencia de rendimiento y concurrencia. Para mostrar el no-show y la asistencia en vivo, se preparan reservas a horas calculadas antes de la corrida.

La primera corrida se hizo contra producción el 06/11/2026, a partir de las 21:00. Se ejecutaron 32 pasos sin errores; siete pasos (8, 9, 19, 20, 24, 38 y 39) no se ejecutaron ese día, porque no se creó una de las reservas de las que dependían. Un primer registro los daba por ejecutados; al revisarlo se corrigió y se hizo una corrida complementaria de esos siete pasos el 11/11/2026, también sin errores. Los montos verificados en producción coinciden con las reglas:

- Cancelación con más de 24 horas: penalización de C$ 0 y reembolso de C$ 600.
- Cancelación con menos de 24 horas: penalización de C$ 300 y reembolso de C$ 300.
- No-show: penalización de C$ 600, solo con el cargo y sin reembolso.
- Baja de una cancha de fútbol campo con una reserva: reembolso de C$ 1,200.

6.3 Tiempo de respuesta (RNF04)

El tiempo de la consulta de disponibilidad se midió contra producción el 05/11/2026 con un script que repite la consulta de forma secuencial: 75 consultas en 5 rondas, todas correctas. El mínimo fue 353 ms, el promedio 369 ms, la mediana 363 ms, el percentil 95 395 ms y el máximo 544 ms, muy

por debajo del límite de 3 segundos. El primer acceso tras la pausa del servicio y de la base queda fuera del criterio de "uso normal" y se informó aparte: 803 ms en esa medición. El diseño ayuda: la disponibilidad resuelve todas las canchas con tres consultas a la base, sin una consulta por cancha.

6.4 Reservas simultáneas (RNF09)

Un script envía diez solicitudes simultáneas de reserva sobre el mismo bloque y verifica en la base lo que quedó. En cada ronda el resultado esperado es una reserva creada y nueve rechazos con `HORARIO_OCUPADO`. La prueba se ejecutó el 11/11/2026 con 5 rondas, y todas dieron ese resultado. Se ejecuta solo contra una base local, porque crea y borra datos de prueba; en producción, el guion muestra el efecto para un usuario (dos clientes sobre el mismo bloque), sin carga.

6.5 Verificación de los requerimientos no funcionales

Tabla 11. Verificación de los requerimientos no funcionales

RNF	Cómo se verificó	Resultado
RNF01	Recorrido completo por pantalla en la prueba integral contra producción, sin instrucciones de uso	Cumplido; sin prueba con personas externas
RNF02	Hash bcrypt al registrar; columna de hash en el esquema	Cumplido
RNF03	Control de rol en la API y en el frontend; pruebas automatizadas y pasos del guion	Cumplido
RNF04	Medición contra producción: 75 consultas, máximo 544 ms	Cumplido
RNF05	Revisión de las pantallas en anchos de celular y de escritorio, sin desborde horizontal; paso de vista móvil del guion	Cumplido
RNF06	Capas por módulo en el backend; api, sesión y un script por página en el frontend	Cumplido
RNF07	Despliegue público en Render y Azure SQL, actualizado por despliegue continuo	Cumplido
RNF08	Tipos de cancha en tabla propia; alta de canchas desde la pantalla	Cumplido
RNF09	Doble defensa en la transacción; prueba de 10 solicitudes simultáneas, 5 rondas	Cumplido

Capítulo VII: Trazabilidad y cumplimiento

7.1 Matriz de trazabilidad

La matriz de trazabilidad (`docs/trazabilidad.md`) registra cada compromiso de los tres entregables: requerimientos, casos de uso, modelos UWE, objetivos, decisiones de arquitectura y de desarrollo. Para cada uno indica dónde se cumple, con la evidencia concreta (archivo, prueba o pull request), y su estado: cumplido, modificado con justificación u omitido con justificación. Se actualizó con cada pull request integrado, hasta que ningún compromiso quedó pendiente.

Tabla 12. Resumen de la matriz de trazabilidad al cierre

Estado	Compromisos	Proporción
Cumplido	182	90,1 %
Modificado	18	8,9 %
Omitido	2	1,0 %
Pendiente	0	0 %
Total	202	100 %

7.2 Compromisos modificados

Los 18 compromisos modificados se cumplieron de otra forma de la escrita, por una decisión justificada. Se agrupan en cinco temas (Tabla 13).

Tabla 13. Compromisos modificados, agrupados por tema

Tema	Qué cambió	Motivo
Pago y estados (6)	La reserva nace PAGADA; no existen "pendiente" ni "reservado"; se agrega NO_SHOW; Simular pago es inclusión y no extensión	El pago simulado es inmediato y ocurre en la misma transacción que la reserva
Modelo de contenido (7)	Usuario en una tabla; TipoCancha, Pago, Auditoria y Notificacion nuevas; Horario por día; Reserva sin asociación a Horario; sin métodos en las entidades; dominio de más de seis entidades	Diseño del esquema; reglas en la capa de servicio
Presentación (1)	El catálogo muestra los bloques libres y no un "estado" por cancha	La disponibilidad se calcula por bloque
Arquitectura (2)	SQL escrito a mano, sin ORM; frontend servido por el mismo servidor	Control de los bloqueos de RNF09; un solo origen y un solo despliegue
Organización (2)	Rotación fija de revisores reemplazada	Disponibilidad del equipo (#31 y

Tema	Qué cambió	Motivo
	por el primero disponible; reparto ajustado	#35)

7.3 Compromisos omitidos y recortes

Dos compromisos se omitieron por decisión del equipo, registrada en el issue del plan:

- **Modelo de presentación de las demás pantallas:** solo existe el del catálogo y la confirmación. Las pantallas siguen el modelo navegacional y se validaron en la prueba integral.
- **Aviso de la regla de penalización al confirmar la reserva:** la regla se muestra al cancelar, con el porcentaje y los montos exactos.

El plan del sprint final definió además un orden de recorte por si el tiempo no alcanzaba. Se aplicaron dos de los tres recortes: no se implementó una observación menor sobre el filtro de la auditoría, y no se hizo una prueba de usabilidad con personas externas; RNF01 quedó respaldado por la prueba integral. El primer recorte, la pantalla de auditoría, no hizo falta: se implementó. No se recortó ningún requerimiento funcional, ni la contenedorización, la integración o el despliegue continuos.

7.4 Cumplimiento de los objetivos

Tabla 14. Cumplimiento de los objetivos específicos

Objetivo específico	Cómo se cumplió	Estado
1. Modelar requerimientos con casos de uso y modelo de contenido	Modelos del Primer Entregable, corregidos (secciones 1.4 y 1.5)	Cumplido
2. Diseñar los modelos navegacional y de presentación por rol	Modelo navegacional de 13 nodos; presentación de dos pantallas	Cumplido en parte: presentación incompleta
3. Especificar el modelo de procesos, con estados y penalización	Diagrama de estados nuevo, flujos de reservar y de baja (sección 1.8)	Cumplido
4. Seleccionar una arquitectura	Monolito en capas (Capítulo II), aplicado por módulo	Cumplido
5. Implementar el backend	28 endpoints con disponibilidad, reservas, pagos simulados y penalizaciones (Capítulo IV)	Cumplido

Objetivo específico	Cómo se cumplió	Estado
6. Implementar el frontend por rol	14 páginas con menú por rol (sección 4.6)	Cumplido
7. Desplegar y documentar	Producción pública con despliegue continuo; documentación de despliegue y versiones (Capítulo V)	Cumplido

Conclusiones

- **El sistema cumple lo comprometido.** Los 17 requerimientos funcionales y los 9 no funcionales se cumplieron, RF06 con una modificación justificada, y la matriz de trazabilidad cerró con 202 compromisos sin pendientes: 182 cumplidos, 18 modificados con justificación y 2 omitidos con justificación.
- **La arquitectura elegida en el Segundo Entregable fue adecuada.** El monolito en capas por módulo permitió el trabajo en paralelo de tres personas y resolvió la regla más delicada, la concurrencia de reservas, con una transacción y un índice sobre una sola base; la prueba con diez solicitudes simultáneas lo confirmó.
- **Separar las reglas de negocio en funciones puras pagó.** Penalizaciones y transiciones de estado se probaron en sus bordes exactos sin base de datos, y los montos verificados en producción coincidieron con ellas.
- **Los modelos UWE se mantuvieron vivos.** La implementación reveló contradicciones en los modelos del Primer Entregable, que se corrigieron y documentaron en lugar de dejar que modelo y sistema divergieran.
- **La integración y el despliegue continuos sostuvieron la calidad bajo presión de tiempo.** Con 710 pruebas, prueba de humo de la imagen y revisión obligatoria en cada cambio, el sistema llegó a producción por un flujo automatizado y repetible, sin costo de infraestructura.
- **La trazabilidad hizo verificable el cumplimiento.** Registrar cada compromiso con su evidencia permitió saber en todo momento qué faltaba y justificar cada cambio, como pidió el docente.

Recomendaciones y trabajo futuro

Quedan limitaciones conocidas que un uso real del sistema debería atender:

- **Pruebas con la base de datos en la integración continua:** las pruebas automatizadas no ejercitan el SQL. Un servicio de SQL Server en el flujo de integración permitiría correr la prueba de concurrencia y pruebas de los repositorios en cada cambio.
- **Usabilidad con usuarios reales:** la interfaz se validó con el recorrido del equipo. Una prueba con clientes y personal del complejo, junto con el modelo de presentación completo, orientaría mejoras de la interfaz, que hoy es funcional y sobria.
- **No-show programado:** las reservas vencidas se marcan cuando alguien opera el sistema. Una tarea periódica haría que la notificación de no-show llegue a tiempo aunque nadie use el sistema.
- **Infraestructura:** el plan gratuito pausa el servicio y la base cuando no se usan. Para operar de verdad, conviene un plan sin pausa y probar el procedimiento de rollback, que está documentado pero no ejercitado.
- **Funciones nuevas:** una pasarela de pago real, avisos por correo o mensajería, y la migración gradual planteada en el Capítulo II si el complejo creciera.

Anexos

Anexo A. Enlaces y documentos

Tabla 15. Enlaces y documentos del proyecto

Recurso	Ubicación
Repositorio	https://github.com/Mattmnz/reservas-zacarias-guerra
Sistema en producción	https://reservas-zacarias-guerra.onrender.com
Descripción, ejecución y forma de trabajo	README.md
Matriz de trazabilidad	docs/trazabilidad.md
Modelos UWE corregidos y sus fuentes	docs/modelos/
Guion de demostración y registro de corridas	docs/guion-demo.md
Despliegue y control de versiones	docs/despliegue.md
Peticiones de prueba por módulo	docs/http/
Frontend: rutas, convenciones y componentes	public/README.md
Plan del sprint final y decisiones	Issue #35 del repositorio